Univerzitet u Sarajevu
Elektrotehnički fakultet
Predmet: Uvod u digitalnu obradu slike
Predmetni profesor: prof. dr Emir Sokić
Autor: Berina Biberović
Akademska godina: 2022/2023.

Tutorijal 3

Operacije nad slikom u prostornom domenu

Procesiranje slike u prostornom domenu

Procesiranje monohromatske slike u prostornom domenu predstavlja primjenu operatora T nad datom slikom $f(x, y)$ u domenu (x, y) kako bi se dobila rezultantna slika $g(x, y)$. Možemo razlikovati dva tipa procesiranja slike u prostornom domenu - transformacija vrijednosti i transformacija korištenjem maske.

Transformacija vrijednosti

Transformacija vrijednosti (ili drugim imenom mapiranje intenziteta) podrazumijeva da operator za izračunavanje vrijednosti zahtjeva okolinu dimenzija 1×1 - operator se ne primjenjuje na širu okolinu koordinate (x, y) već samo na jedan piksel. Kao primjer navest ćemo istezanje kontrasta (eng. *contrast streching*) ili funkcije praga (eng. *thresholding*). Ovo je i tzv. tačkasto procesiranje. Jednostavnije rečeno, kod ovakvih transformacija slika svaka vrijednost piksela na izlaznoj slici zavisi od odgovarajućeg piksela na ulaznoj slici (plus možda i od globalih informacija i parametara).

Podešavanje osvjetljenja i kontrasta slike

Prvi primjer koji ćemo obraditi je podešavanje osvjetljenja i kontrasta slike. Dva često korištena tačkasta procesiranja koja se koriste su množenje i sabiranje sa konstantnom:

$$g(x) = \alpha f(x) + \beta$$

Parametri $\alpha > 0$ i β se zovu *gain* i *bias* parametri - ovi parametri kontrolišu kontrast i osvjetljenje, respektivno. U gornjoj jednačini $f(x)$ predstavlja sliku na ulazu, dok $g(x)$ predstavlja izlaznu sliku. Može se onda pisati gornja jednačina kao:

$$g(i, j) = \alpha \cdot f(i, j) + \beta$$

gdje i označava red piksela, a j kolonu piksela na slici. Slijedi jednostavan primjer gdje se pikseli originalne slike transformiraju na način kako je prethodno objašnjeno.

```
#include <opencv4/opencv2/highgui.hpp>
#include <opencv4/opencv2/core/mat.hpp>
#include <opencv4/opencv2/core/saturate.hpp>

int main() {

    // Ucitavanje originalne slike:
    cv::Mat original = cv::imread("../maca.jpeg");

    // Kreiranje nove slike:
```

```
11      cv::Mat nova = cv::Mat::zeros(original.size(), original.type());
12
13      // Parametri alpha i beta:
14      double alpha = 3;
15      int beta = 25;
16
17      // Transformiranje svakog piksela g(i, j) = alpha * f (i, j) + beta:
18      for(int i = 0; i < original.rows; i++) {
19          for(int j = 0; j < original.cols; j++) {
20              // Slika original je tipa BGR, tako da je i nova tipa BGR -
                   imamo tri kanala za svaki element!
21              for(int k = 0; k < original.channels(); k++) {
22                  nova.at<cv::Vec3b>(i, j)[k] =
                       cv::saturate_cast<uchar>(alpha *
                       original.at<cv::Vec3b>(i, j)[k] + beta);
23              }
24          }
25      }
26
27      // Prikaz originalne slike i nove slike:
28      cv::imshow("Originalna slika", original);
29      cv::imshow("Nova slika", nova);
30
31      cv::waitKey();
32
33      return 0;
34  }
```

Nove funkcije sa kojima se susrećemo u prethodnom isječku koda su sljedeće: `cv::Mat::zeros(cv::Size vel, int tip)` funkcija koja pravi niz nula specificirane veličine i tipa, gdje parametar `vel` predstavlja veličinu izlazne slike, a parametar `tip` predstavlja tip koji želimo da izlazna slika bude; `static _Tp cv::saturate_cast (uchar v)` koja predstavlja šablonsku funkciju za tačnu konverziju jednog primitivnog tipa u drugi - u ovom slučaju smo iskoristili ovu funkciju kako bismo transformaciju množenja i sabiranja mogli korektno konvertovati u potrebni tip koji se može ubaciti u novu sliku koju smo kreirali (kako je `alpha` tipa `double`, to vrijednost transformacije može biti necjelobrojna, tako da uz pomoć navedene funkcije postižemo da je rezultat transformacije validan).

Preporučuje se studentima da mijenjaju parametre `alpha` i `beta` kako bi vidjeli na koji način se mijenja slika kako podešavamo parametre kontrasta i osvjetljenja.

Bitno je istaći da prethodni kod se može optimizirati ukoliko se koristi funkcija `cv::convertTo`. Dovoljno je napisati sljedeće umjesto `for` petlji u linijama 18 do 25: `original.convertTo(nova, -1, alpha, beta);`. Funkcija `void cv::Mat::convertTo(OutputArray m, int rtip, double alpha, double beta = 0)const` konvertuje niz u drugi tip sa željenim skaliranjem. U našem slučaju je željeni tip izlazne slike isti kao ulazne, tako da za parametar `rtip` stavljamo `-1`. Još jedna napomena za ovu funkciju - ukoliko parametar `m` nema odgovarajuću veličinu prije ove operacije, onda se taj paramear realocira.

Gama korekcija

Gama korekcija se može koristiti kako bi se korektovala svjetlost slike ali korištenjem nelinerane transformacije između ulaznih i izlaznih vrijednosti:

$$Izlaz = \left(\frac{Ulaz}{255}\right)^{\gamma} \times 255$$

Kako je ova relacija nelinearna, efekat neće biti isti za sve piksele i zavisit će od njihove ulazne vrijednosti. Ako je $\gamma < 1$ onda će ulazni tamniji regioni biti svjetliji, ali obratno vrijedi

za $\gamma > 1$.

U nastavku slijedi primjer korištenja linearne i gama transformacije nad istom slikom koja je dijelom zamračena, kako bismo vidjeli razliku između gama i linearne transformacije.

```cpp
#include <opencv4/opencv2/highgui.hpp>
#include <opencv4/opencv2/core.hpp>
#include <opencv4/opencv2/imgproc.hpp>

int main() {

    // Ucitavanje originalne slike:
    cv::Mat original = cv::imread("../maca2.jpg");
    cv::resize(original, original, cv::Size(), 0.2, 0.2);

    // Kreiranje nove slike:
    cv::Mat linearna = cv::Mat::zeros(original.size(), original.type());

    // Parametri alpha i beta:
    double alpha = 1.3;
    int beta = 40;

    // Transformiranje svakog piksela g(i, j) = alpha * f (i, j) + beta:
    original.convertTo(linearna, -1, alpha, beta);

    // Sada koristimo gamma transformaciju:
    double gamma = 0.35;
    cv::Mat lookUpTable(1, 256, CV_8U);
    uchar *p = lookUpTable.ptr();
    for(int i = 0; i < 256; i++)
        p[i] = cv::saturate_cast<uchar>(cv::pow((i / 255.0), gamma) *
            255.0);

    cv::Mat gama_transf = original.clone();
    cv::LUT(original, lookUpTable, gama_transf);

    // Prikaz originalne slike i nove slike:
    cv::imshow("Originalna slika", original);
    cv::imshow("Linerana transformacija", linearna);
    cv::imshow("Gama transformacija", gama_transf);

    cv::waitKey();

    return 0;
}
```

Možemo primijetiti da u ovom primjeru koristimo *look-up* tabelu za kalkulaciju gama transformacije - to je da ubrzamo program jer je potrebno samo jednom izračunati 256 vrijednosti unutar *look-up* tabele. Slično kao za lineranu transformaciju u prethodnom primjeru, možemo vidjeti da svaki elemenat *look-up* tabele transformiramo preko gama transformacije. Nakon toga, pravimo novu sliku `gama_transf` koja je kopija originalne. Kada to uradimo, onda koristimo funkciju `void cv::LUT(InputArray ulaz, InputArray lookuptabela, OutputArray izlaz)` kako bismo primijenili transformaciju iz *look-up* tabele na ulazni niz. Ono što ova funkcija uradi je da izlazni niz popuni vrijednostima iz *look-up* tabele. Indeksi unosa se uzimaju iz ulaznog niza.

Na kraju svega prikazujemo sve tri slike - originalnu, linerarno transformisanu sliku i sliku dobijenu nakon gama transformacije koje su prikazane na slikama 1, 2 i 3.

Kao što se može vidjeti, gama transformacija je bolje osvijetlila tamni dio slike, dok je linearna transformacija podjednako osvijetlila cijelu sliku. Sada vidimo bolje na slici 3 pozadinu koja je tamna na originalnoj slici.



Slika 1: Originalna slika koja je skalirana



Slika 2: Slika nakon linearne transformacije



Slika 3: Slika nakon gama transformacije

Gama korekcija bi trebala da daje manji efekat zasićenosti jer je mapiranje nelinearno i ne postoji numerička zasićenost kao u linearnoj metodi (ukoliko vrijednost padne ispod 0, zaokružuje se na 0, a ukoliko pređe 255 zaokružuje se na 255 u linearnoj metodi). Možemo više uporediti ove tri slike ukoliko im isplotamo histograme, što će biti pokazano u sljedećem dijelu tutorijala.

Histogrami slika

Histogrami su skupljeni brojevi podataka koji su organizirani u set predefiniranih *binova*. U slučaju histograma slika, u ovom tutorijalu govorimo o broju različitih intenziteta piksela na jednoj slici, ali možemo tražiti histogram bilo čega što nas zanima na slici, ne samo intenziteta piksela.

U sljedećem primjeru ćemo izračunati histograme prethodne tri slike 1, 2 i 3.

```
#include <opencv4/opencv2/highgui.hpp>
#include <opencv4/opencv2/core.hpp>
#include <opencv4/opencv2/imgproc.hpp>
#include <opencv4/opencv2/imgcodecs.hpp>
#include <opencv4/opencv2/core/fast_math.hpp>
#include <vector>

int main() {

    // Ucitavanje slike kojoj trazimo histogram:
    cv::Mat slika = cv::imread("../original.jpg", cv::IMREAD_COLOR);

    // Razdvajanje slike na 3 kanala (B, G i R):
    std::vector<cv::Mat> kanali;
    cv::split(slika, kanali);

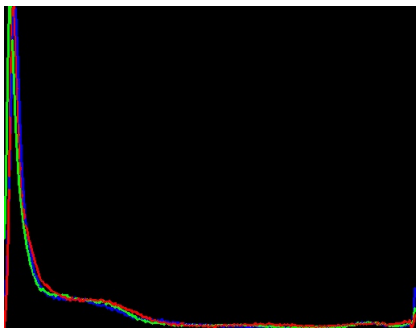
    // Kako radimo sa B, G i R prostorima, znamo da su njihove vrijednosti
       od 0 do 255:
    int histSize = 256;
    float range[] = {0, 256};
    const float* histRange[] = {range};

    cv::Mat b_hist, g_hist, r_hist;

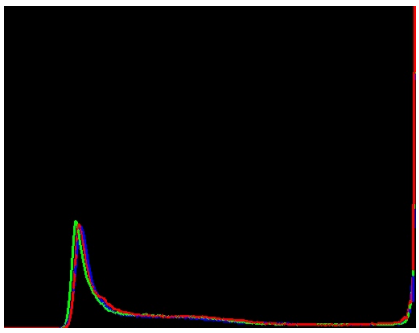
    // Postoji funkcija u OpenCV-u koja nam racuna histogram:
    cv::calcHist(&kanali[0], 1, 0, cv::Mat(), b_hist, 1, &histSize,
       histRange, true, false);
    cv::calcHist(&kanali[1], 1, 0, cv::Mat(), g_hist, 1, &histSize,
       histRange, true, false);
```

```
27      cv::calcHist(&kanali[2], 1, 0, cv::Mat(), r_hist, 1, &histSize,
            histRange, true, false);
28
29      // Treba napraviti sliku koja ce prikazati histograme za B, G i R:
30      int hist_sirina = 512, hist_visina = 400;
31      int bin_sirina = cvRound((double) hist_sirina / histSize );
32      cv::Mat histogramSlike(hist_visina, hist_sirina, CV_8UC3, cv::Scalar(0,
            0, 0));
33
34      // Prije iscrtavanja histograma, potrebno je normalizirati svaki
            histogram tako da vrijednosti
35      // upadaju u opseg parametara koje smo naveli:
36      cv::normalize(b_hist, b_hist, 0, histogramSlike.rows, cv::NORM_MINMAX,
            -1, cv::Mat());
37      cv::normalize(g_hist, g_hist, 0, histogramSlike.rows, cv::NORM_MINMAX,
            -1, cv::Mat());
38      cv::normalize(r_hist, r_hist, 0, histogramSlike.rows, cv::NORM_MINMAX,
            -1, cv::Mat());
39
40      // Crtanje histograma tako sto pristupamo binovima i crtamo vrijednosti:
41      for(int i = 1; i < histSize; i++) {
42          cv::line(histogramSlike, cv::Point(bin_sirina * (i - 1),
                hist_visina - cvRound(b_hist.at<float>(i - 1))),
43                   cv::Point(bin_sirina * (i), hist_visina -
                         cvRound(b_hist.at<float>(i))), cv::Scalar(255, 0, 0),
                         2, 8, 0);
44          cv::line(histogramSlike, cv::Point(bin_sirina * (i - 1),
                hist_visina - cvRound(g_hist.at<float>(i - 1))),
45                   cv::Point(bin_sirina * (i), hist_visina -
                         cvRound(g_hist.at<float>(i))), cv::Scalar(0, 255, 0),
                         2, 8, 0);
46          cv::line(histogramSlike, cv::Point(bin_sirina * (i - 1),
                hist_visina - cvRound(r_hist.at<float>(i - 1))),
47                   cv::Point(bin_sirina * (i), hist_visina -
                         cvRound(r_hist.at<float>(i))), cv::Scalar(0, 0, 255),
                         2, 8, 0);
48      }
49
50      // Prikaz slike i histograma te slike:
51      cv::imshow("Slika", slika);
52      cv::imshow("Histogram", histogramSlike);
53
54      cv::waitKey();
55
56      return 0;
57  }
```

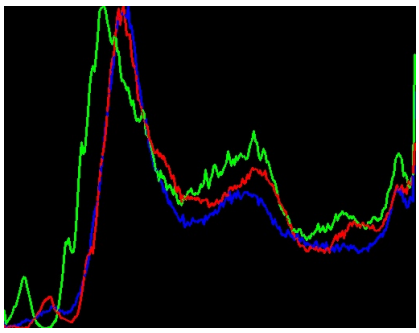
Kako bismo izračunali histogram slike koja je u BGR formatu učitana, bilo je potrebno prvo podijeliti na njena tri kanala korištenjem funkcije `cv::split`. Postoji funkcija u OpenCV-u koja nam omogućuje računanje histograma `cv::calcHist` kojoj smo proslijedili odgovarajuće parametre (naznačeno je u komentarima koda zašto se neke vrijednosti prosljeđuju). Bilo je onda potrebno napraviti novi objekat tipa `cv::Mat` koji će zapravo biti slika koja je histogram - naznačili smo njenu veličinu, tip i boju kojom će svi pikseli biti inicijalizirani (crna). Nakon toga uz pomoć funkcije `cv::normalize` normaliziramo histogram svakog kanala tako da se može fino upisati u novonapravljeni objekat `histogramSlike`. Ucrtavanje histograma radimo preko funkcije `cv::line` koja nam crta linije tako što povezujemo vrijednost iz trenutnog elementa niza `b_hist`, `g_hist` i `r_hist` sa prethodnim. Dobiveni histogrami slika 1, 2 i 3 su prikazani na slikama 4, 5 i 6, respektivno.



Slika 4: Histogram originalne slike



Slika 5: Histogram slike nakon linearne transformacije



Slika 6: Histogram slike nakon gama transformacije

Ono što možemo očitati je da kod originalne slike veliki dio vrijednosti piksela je u tamnom dijelu. Linearnom transformacijom smo postigli pomjeranje vrijednosti piksela više ka središnjem dijelu, ali opet vidimo da ima *peak* prema tamnijem dijelu, iako nije toliko velik kao kod originala. Gama transformacijom smo uspjeli da vrijednosti piksela otprilike izjednačimo tako da više nisu u tamnijem dijelu (što smo i naveli da će se desiti prilikom gama transformacije).

Zadatak 1

Uslikati (ili naći na internetu) sliku koja je previše svijetla. Uraditi linearnu i gama transformaciju nad njom, napraviti histograme slika i komentarisati rezultate koji su dobiveni.

Izjednačavanje histograma u OpenCV-u

Izjednačavanje histograma predstavlja metodu koja popravlja kontrast slike, kako bi se razvukao opseg intenziteta. Kako bismo što bolje shvatili ovaj koncept, najbolje je da ga pokažemo na primjeru.

```cpp
#include <opencv4/opencv2/highgui.hpp>
#include <opencv4/opencv2/core.hpp>
#include <opencv4/opencv2/imgproc.hpp>

int main() {

    // Ucitavanje slike:
    cv::Mat slika = cv::imread("../original.jpg", cv::IMREAD_COLOR);

    // Treba nam grayscale slika:
    cv::Mat grayscale;
    cv::cvtColor(slika, grayscale, cv::COLOR_BGR2GRAY);

    // Sad trebamo da izjednacimo histogram nad ovom slikom:
    cv::Mat izjednacena;
    cv::equalizeHist(grayscale, izjednacena);

    cv::imshow("Grayscale", grayscale);
    cv::imshow("Nakon izjednacenja histograma", izjednacena);

    cv::waitKey();

    return 0;
}
```

Prethodni kod pokazuje kako možemo uz pomoć OpenCV funkcije `cv::equalizeHist` da izjednačimo histogram *grayscale* slike. Ova funkcija prima dva parametra - originalnu sliku u *grayscale*-u i izlaznu sliku u koju se stavlja rezultat izjednačenja histograma (oba parametra su tipa `cv::Mat`).

Prethodni program smo izvršili nad istom slikom kao maloprije i dobili smo rezultat koji prikazan na slikama 7 i 8.



Slika 7: Originalna *grayscale* slika

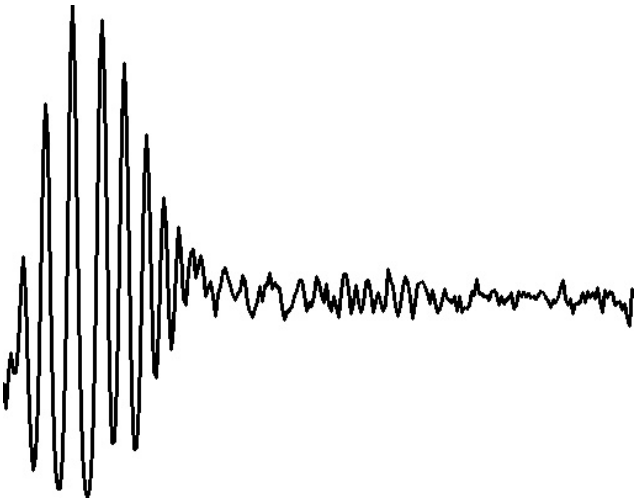


Slika 8: Slika nakon izjednačenja histograma

Ukoliko iskoristimo prethodni kod da izračunamo histograme ovih slika (bez da ih razbijamo na 3 kanala jer su obje grayscale), dobijamo histograme koji su prikazani na slikama 9 i 10.



Slika 9: Histogram originalne *grayscale* slike



Slika 10: Histogram slike nakon izjednačenja histograma

Zadatak 2

Komentarisati dobivene histograme iz prethodnog primjera. Šta smo dobili?

Napomene

Korištena tema za prikaz C++ koda na tutorijalima je `solarized`[1].

Literatura

[1] Ethan Schoonover. *Solarized*. `http://ethanschoonover.com/solarized`. Pristup stranici: 20. maj 2023. 2011.